# NMAM Institute of Technology, Nitte

Department of Computer Science and Engineering

# Operating Systems Laboratory

## LABORATORY MANUAL

Academic Year 2026–27

| | |
|---|---|
| Name | : Prajwal Jogi |
| USN | : |
| Semester / Section | : |

# INDEX


| Expt. No. | Date | Title of the Experiment | Page No. | Signature |
|---|---|---|---|---|
| 1 | 12/08/2026 | exec() system call | 1 | |
| 2 | 12/08/2026 | File read using system calls | 2 | |
| 3 | 19/08/2026 | Child processes using fork() | 3 | |
| 4 | 26/08/2026 | File write using system calls | 4 | |
| 5 | 02/09/2026 | Signal handling | 5 | |
| 6 | 09/09/2026 | FCFS scheduling | 7 | |
| 7 | 30/09/2026 | SJF scheduling | 9 | |
| 8 | 30/09/2026 | Round Robin scheduling | 11 | |

# Experiment No. 1
## exec() system call

**Aim:** Write a C program using exec() system call:

a) Read the name of the shell script from the keyboard

b) Execute that script using /bin/sh

c) Display an appropriate error message if the script cannot be found or executed

d) The shell script should prompt the user to enter the 3 numbers, find and display the greatest of 3 numbers

### Program

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    char sname[100];
    printf("Enter shell script name: ");
    scanf("%99s", sname);
    /* the script must exist and be readable */
    if(access(sname, R_OK) == -1) {
        perror("Error: cannot open script");
        return 1;
    }
    if(execl("/bin/sh", "sh", sname, (char*)NULL) == -1)
        perror("Error: cannot execute script");

    return 0;
}
```

### Shell script: largest.sh

```bash
#!/bin/bash
read -p "Enter 3 numbers: " a b c
if [ $a -ge $b ] && [ $a -ge $c ]
then
    echo "$a is the greatest"
elif [ $b -ge $a ] && [ $b -ge $c ]
then
    echo "$b is the greatest"
else
    echo "$c is the greatest"
fi
```

### Output

```
gcc main.c
./a.out
Enter shell script name: largest.sh
Enter 3 numbers: 3 4 2
4 is the greatest

gcc main.c
./a.out
Enter shell script name: large.sh
Error: cannot open script: No such file or directory
```

# Experiment No. 2

## File read using system calls

**Aim:** Write a C program that uses appropriate unique system calls to read the contents of the file and display it on the terminal. The program should prompt the user to enter the filename as input from the keyboard.

### Program

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

int main()
{
    char fname[100], buffer[100];
    int fd;
    ssize_t n;
    printf("Enter filename: ");
    scanf("%99s", fname);
    fd = open(fname, O_RDONLY);
    if (fd == -1)
    {
        perror("Error");
        return 1;
    }
    printf("file contents:\n");
    /* flush stdio before using write() directly */
    fflush(stdout);
    /* keep reading until end of file */
    while ((n = read(fd, buffer, sizeof(buffer))) > 0)
    {
        if (write(STDOUT_FILENO, buffer, n) != n)
        {
            perror("Error");
            break;
        }
    }
    if (n == -1)
        perror("Error");
    close(fd);
    return 0;
}
```

### Output

```
gcc main.c
./a.out
Enter filename: contents.txt
file contents:
Hello World

gcc main.c
./a.out
Enter filename: cont.txt
Error: No such file or directory
```

# Experiment No. 3
## Child processes using fork()

**Aim:** Write a C program to create five child processes using the system call fork() and display their ids.

### Program

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int i;
    pid_t pid;
    for(i=1; i<=5; i++){
        /* flush so the child does not copy unflushed output */
        fflush(stdout);
        pid = fork();
        if(pid < 0)
        {
            printf("Error ");
            return 1;
        } else if(pid == 0) {
            printf("Child %d with id %d and Parent Process id %d\n",
                   i, getpid(), getppid());
            exit(0);
        } else {
            printf("Parent Created Child %d with pid %d\n",
                   i, pid);
        }
    }
    for(i=1; i<=5; i++)
    {
        wait(NULL);
    }
    printf("Parent pid %d after finishing all Child execution\n",
           getpid());
    return 0;
}
```

### Output

```
gcc main.c
./a.out
Parent Created Child 1 with pid 414
Child 1 with id 414 and Parent Process id 413
Parent Created Child 2 with pid 415
Child 2 with id 415 and Parent Process id 413
Parent Created Child 3 with pid 416
Child 3 with id 416 and Parent Process id 413
Parent Created Child 4 with pid 417
Child 4 with id 417 and Parent Process id 413
Parent Created Child 5 with pid 418
Child 5 with id 418 and Parent Process id 413
Parent pid 413 after finishing all Child execution
```

# Experiment No. 4

## File write using system calls

**Aim:** Write a C program using unique system calls to open a file for writing (create it if it does not exist). The program should read text from the keyboard and write it into the file until Ctrl+D.

### Program

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd;
    char buffer[1000], fname[100];
    printf("Enter the filename to be written \n");
    scanf("%99s", fname);
    fd = open(fname, O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd == -1)
    {
        printf("Open error");
        return 0;
    }
    printf("Enter the contents to be written \n");
    while(1)
    {
        int n = read(0, buffer, sizeof(buffer));
        if(n > 0) {
            write(fd, buffer, n);
        }
        else if(n == 0) {
            break;
        }
        else {
            printf("Read error");
            close(fd);
            return 0;
        }
    }
    close(fd);
    printf("Contents written to %s\n", fname);
    return 0;
}
```

### Output

```
gcc main.c
./a.out
Enter the filename to be written
contents.txt
Enter the contents to be written
hello world
Contents written to contents.txt

cat contents.txt
hello world
```

# Experiment No. 5
## Signal handling

**Aim:** Write a C program that registers a signal handler for SIGINT, SIGTERM, SIGHUP, SIGALRM, SIGUSR1. The handler should receive the signal and perform some operation.

### Program

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void signalhandler(int sig)
{
    if(sig == SIGINT)
    {
        printf("Caught an Interrupt Signal\n");
    }
    if(sig == SIGTERM)
    {
        printf("Caught a Terminate Signal\n");
        exit(0);
    }
    if(sig == SIGHUP){
        printf("Caught a Hangup Signal\n");
    }
    if(sig == SIGUSR1)
    {
        printf("Caught a User Signal\n");
    }
    if(sig == SIGALRM)
    {
        printf("Caught an Alarm Signal\n");
    }
}

int main()
{
    signal(SIGINT, signalhandler);
    signal(SIGTERM, signalhandler);
    signal(SIGHUP, signalhandler);
    signal(SIGALRM, signalhandler);
    signal(SIGUSR1, signalhandler);
    alarm(20);
    printf("Process Id of program running = %d\n", getpid());
    while(1)
    {
        printf("Waiting for signal\n");
        sleep(5);
    }
    return 0;
}
```

## Output

***Terminal 1***

```
gcc main.c
./a.out
Process Id of program running = 430
Waiting for signal
Caught an Interrupt Signal
Waiting for signal
Caught a Hangup Signal
Waiting for signal
Caught a User Signal
Waiting for signal
Caught an Alarm Signal
Waiting for signal
Caught a Terminate Signal
```

***Terminal 2***

```
kill -SIGINT 430
kill -SIGHUP 430
kill -SIGUSR1 430
kill -SIGALRM 430
kill -SIGTERM 430
```

# Experiment No. 6
## FCFS scheduling

**Aim:** Write a C program to implement First Come First Serve scheduling algorithm to determine total waiting time and average waiting time, total turn around time and average turn around time.

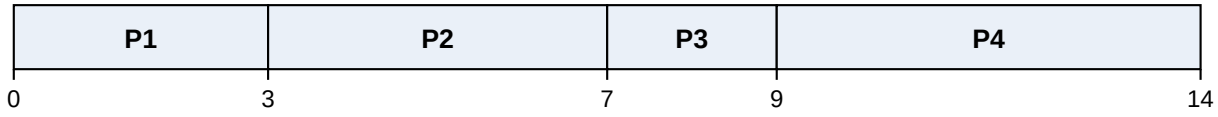
### Program

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

struct Process
{
    int pid, bt, wt, tat;
};

int main()
{
    int n, i;
    double awt = 0, atat = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    if(n <= 0)
    {
        printf("Number of processes must be positive\n");
        return 1;
    }
    struct Process P[n];
    for(i=0; i<n; i++)
    {
        P[i].pid = i+1;
        printf("Enter burst time for P%d: ", i+1);
        scanf("%d", &P[i].bt);
    }
    P[0].wt = 0;
    P[0].tat = P[0].bt + P[0].wt;
    for(i=1; i<n; i++)
    {
        P[i].wt = P[i-1].wt + P[i-1].bt;
        P[i].tat = P[i].bt + P[i].wt;
    }
    printf("\n Process BT\t WT\t TAT\t");
    for(i=0; i<n; i++)
    {
        printf("\n %d\t %d\t %d\t %d ", P[i].pid, P[i].bt, P[i].wt,
               P[i].tat);
    }
    for(i=0; i<n; i++)
    {
        awt += P[i].wt;
        atat += P[i].tat;
    }
    printf("\n\nTotal Waiting Time = %.2f\n", awt);
    printf("Average Waiting Time = %.2f\n", awt/n);
    printf("Total Turn Around Time = %.2f\n", atat);
    printf("Average Turn Around Time = %.2f\n", atat/n);
    return 0;
}
```

## Output

```
gcc main.c
./a.out
Enter number of processes: 4
Enter burst time for P1: 3
Enter burst time for P2: 4
Enter burst time for P3: 2
Enter burst time for P4: 5

 Process BT      WT      TAT
 1       3       0       3
 2       4       3       7
 3       2       7       9
 4       5       9       14

Total Waiting Time = 19.00
Average Waiting Time = 4.75
Total Turn Around Time = 33.00
Average Turn Around Time = 8.25
```

## Gantt chart



# Experiment No. 7
## SJF scheduling

**Aim:** Write a C program to implement Shortest Job First (SJF) scheduling algorithm to determine total waiting time, average waiting time, total turn around time, average turn around time.

### Program

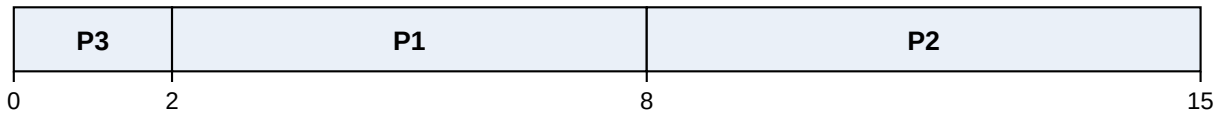
```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

struct Process
{
    int pid, bt, wt, tat;
};

int main()
{
    int n, min;
    double awt=0, atat=0;
    printf("Enter no of process: ");
    scanf("%d", &n);
    if(n <= 0)
    {
        printf("Number of processes must be positive\n");
        return 1;
    }
    struct Process p[n], temp;
    for(int i=0; i<n; i++)
    {
        p[i].pid = i+1;
        printf("Enter burst time for p%d : ", i+1);
        scanf("%d", &p[i].bt);
    }
    for(int i=0; i<n; i++)
    {
        min = i;
        for(int j=i+1; j<n; j++)
        {
            /* equal burst times: keep arrival (pid) order */
            if(p[j].bt < p[min].bt ||
               (p[j].bt == p[min].bt && p[j].pid < p[min].pid))
            {
                min = j;
            }
        }
        temp = p[i];
        p[i] = p[min];
        p[min] = temp;
    }
    p[0].wt = 0;
    p[0].tat = p[0].bt + p[0].wt;
    for(int i=1; i<n; i++)
    {
        p[i].wt = p[i-1].wt + p[i-1].bt;
        p[i].tat = p[i].wt + p[i].bt;
    }
    printf("\nProcess BT\t WT\t TAT");
    for(int i=0; i<n; i++)
    {
        printf("\n%d\t %d\t %d\t %d", p[i].pid, p[i].bt,
               p[i].wt, p[i].tat);
```

```
    }
    for(int i=0; i<n; i++)
    {
        awt += p[i].wt;
        atat += p[i].tat;
    }
    printf("\n\nTotal Waiting Time = %.2f\n", awt);
    printf("Total Turn Around Time = %.2f\n", atat);
    printf("Average Waiting Time = %.2f\n", awt/n);
    printf("Average Turn Around Time = %.2f\n", atat/n);
    return 0;
}
```

## Output

```
gcc main.c
./a.out
Enter no of process: 3
Enter burst time for p1 : 6
Enter burst time for p2 : 7
Enter burst time for p3 : 2

Process BT       WT      TAT
3        2       0       2
1        6       2       8
2        7       8       15

Total Waiting Time = 10.00
Total Turn Around Time = 25.00
Average Waiting Time = 3.33
Average Turn Around Time = 8.33
```

## Gantt chart



# Experiment No. 8
## Round Robin scheduling

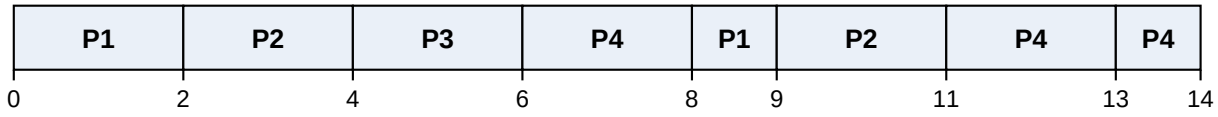
**Aim:** Write a C program to implement Round Robin scheduling algorithm to determine total waiting time, total turn around time, average waiting time, average turn around time.

### Program

```c
#include <stdio.h>

struct Process
{
    int pid, bt, wt, tt;
};

int main()
{
    int i, n, tq, time=0, totwt=0, tottt=0, flag=0;
    float avgwt, avgtt;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    if(n <= 0)
    {
        printf("Number of processes must be positive\n");
        return 1;
    }
    struct Process p[n];
    int remBT[n];
    for(i=0; i<n; i++)
    {
        p[i].pid = i+1;
        printf("Enter burst time for p%d : ", i+1);
        scanf("%d", &p[i].bt);
        remBT[i] = p[i].bt;
    }
    printf("Enter the time quantum: ");
    scanf("%d", &tq);
    if(tq <= 0)
    {
        printf("Time quantum must be positive\n");
        return 1;
    }
    while(flag==0)
    {
        flag=1;
        for(i=0; i<n; i++)
        {
            if(remBT[i] > 0)
            {
                flag=0;
                if(remBT[i] > tq)
                {
                    time = time + tq;
                    remBT[i] = remBT[i]-tq;
                } else {
                    time = time + remBT[i];
                    remBT[i] = 0;
                    p[i].tt = time;
                    p[i].wt = p[i].tt - p[i].bt;
                }
            }
        }
    }
    printf("\nProcess BT\t WT\t TAT ");
```

```
    for(i=0; i<n; i++)
    {
        printf("\n %d\t %d\t %d\t %d ", p[i].pid, p[i].bt,
               p[i].wt, p[i].tt);
        totwt = totwt + p[i].wt;
        tottt = tottt + p[i].tt;
    }
    avgwt = (float) totwt/n;
    avgtt = (float) tottt/n;
    printf("\nTotal Waiting Time: %d ms\nAverage Waiting Time: %.2f ms\n",
           totwt, avgwt);
    printf("Total Turn Around Time: %d ms\nAverage Turn Around Time: %.2f ms\n",
           tottt, avgtt);
    return 0;
}
```

## Output

```
gcc main.c
./a.out
Enter the number of processes: 4
Enter burst time for p1 : 3
Enter burst time for p2 : 4
Enter burst time for p3 : 2
Enter burst time for p4 : 5
Enter the time quantum: 2

Process BT       WT      TAT
 1       3       6       9
 2       4       7       11
 3       2       4       6
 4       5       9       14
Total Waiting Time: 26 ms
Average Waiting Time: 6.50 ms
Total Turn Around Time: 40 ms
Average Turn Around Time: 10.00 ms
```

## Gantt chart